BusyLean: Naming Ever Larger Numbers in Lean

Research notes on constructive large numbers with formal comparisons

BusyLean contributors

October 2026

Abstract

We port Cody Roux's *name the biggest number* contest from Coq to Lean 4. In the contest every number must be a closed, computable definition, and every claim "my number is bigger" must be a kernel-checked theorem. We give an exact port of the incumbent Contender 5 (the largest numeral of a raw System T term of depth 42). We review and rebuild five submitted proposals, and we construct Contender 6, which combines their best ideas: an object language with W-types over every finite type, relativized to oracle families, and reflected along every tree that the language itself can define. Along the way we prove formally that one proposal's re-implementation of the old language agrees with the exact port. We then construct Contender 7: System F with Contender 6 as an oracle. Its strong normalization is proved inside Lean by Girard's reducibility candidates, which is possible because Lean's `Prop` is impredicative, and normalization runs as a certified total function. Contender 8 adds type operators (System F_ω), with candidates lifted to kinds and a sound, fuel-bounded type checker. Contender 9 reflects F_ω along the W-language's trees, and we explain why the Calculus of Constructions is not a growth-rate jump. Contender 10 makes the jump cheaply: kind polymorphism lets one program reach kinds of unbounded order at run time, beyond higher-order arithmetic. A second batch of five proposals is reviewed; two of its ideas, operator-level reflection and ordinal notations above a language-defined index, are adopted in Contender 12. Contender 13 adds a predicative universe of operators, a kind `dyn` interpreted as $\sum_k \mathrm{Sem}(k)$, with level-1 kind variables that range over it. Contender 14 climbs a ladder of such universes: at input n it reflects over the language with n stacked universes, all normalized by one theorem that is parametric in the universe.

Contents

1 The game **2**

2 How to compare unimaginably large numbers **3**

3 Contender 5, exactly **3**

4 Five proposals **4**

5 Contender 6 **4**

5.1 The object language 4

5.2 Oracles for free 4

5.3 Reflection along the language's own trees 4

5.4 What is proved 5

6 Engineering lessons **5**

7 Contender 7: impredicative polymorphism **5**
7.1 Why Lean can host System F . . . 6
7.2 The formalization . . . 6
7.3 From programs to numbers . . . 6
7.4 The contender . . . 6

8 Contender 8: type operators **7**
8.1 Candidates over kinds . . . 7
8.2 Deciding type equality, soundly . . . 7
8.3 The contender . . . 7

9 Contender 9: reflecting F_ω along the W-language's trees **8**
9.1 Why not the Calculus of Constructions . . . 8
9.2 A multiplicative step . . . 8
9.3 Budgets for a growing stack . . . 8

10 Contender 10: kind polymorphism **8**
10.1 Unbounded orders without dependent types . . . 8
10.2 The model . . . 9
10.3 The contender . . . 9

11 Contender 11: reflecting the kind-polymorphic language **9**

12 A second batch of proposals, and Contender 12 **10**
12.1 Five more proposals . . . 10
12.2 Reusing the generic ideas . . . 10

13 Contender 13: a predicative universe of operators **10**
13.1 The idea . . . 10
13.2 The model . . . 11
13.3 The contender . . . 11
13.4 A third batch of proposals . . . 11

14 Contender 14: the ladder of universes **12**
14.1 The idea . . . 12
14.2 One normalization proof for every level . . . 12
14.3 The contender . . . 12

15 Roadmap **13**

16 Status **13**

1 The game

Scott Aaronson's *Who can name the bigger number?* asks for the largest number one can describe concisely. Cody Roux's repository name-the-biggest-number adds two caveats that turn the game into one about *proof*. First, numbers must be *constructive*: computable in principle, with no Busy Beavers. Second, each new contender must come with a *formal proof* that it beats the previous one.

The original contest uses Coq. Its last entry, Contender 5 by Samuel Gruetter, is the largest value of a raw STLC+NatRec term (Gödel's System T) of syntax depth at most 42.

The Lean edition. This repository keeps the contest but switches to Lean 4 (version 4.34.0). The rules (`README.md`) are: closed computable `Nat` definitions; a theorem `contender_(N-1)_lt_contender_N`; the axioms `propext` and `Quot.sound` allowed and `Classical.choice` forbidden; at most 15 s to check the definition and 60 s to check the proof, from a clean build; and no laziness or shenanigans. The axiom policy is deliberate. `Classical.choice` is the only one of Lean's three axioms that conflicts with computability, so it is the one that must go.

2 How to compare unimaginably large numbers

All numbers here are far too large to evaluate. The formal theorems compare them directly, but informal *size* is best discussed with the *fast-growing hierarchy* f_α indexed by ordinals: $f_0(n) = n+1$, $f_{\alpha+1}(n) = f_\alpha^n(n)$, and $f_\lambda(n) = f_{\lambda[n]}(n)$ at limits. A total language L whose programs are all provably terminating in a theory T has its bounded-size maxima growing roughly like $f_{|T|}$, where $|T|$ is the proof-theoretic ordinal of T. Two heuristics organize everything below.

1. **The base language dominates.** System T is calibrated by ε_0. T plus one type of countably branching trees, with recursion into all finite types, is calibrated by the Bachmann–Howard ordinal $\mathrm{BH} = \psi(\varepsilon_{\Omega+1})$. With tree classes at every finite level the calibration is $\psi(\Omega_\omega)$, the ordinal of Π^1_1-CA_0.
2. **Reflection multiplies.** Relativizing a language L to an oracle that grows like f_β yields growth like $f_{\beta+|L|}$. Iterating this along an index of height γ yields $f_{|L|\cdot\gamma}$. Explicit ordinal notations without collapsing functions stay below BH, so a stack of reflections over a weak base cannot catch up with a stronger base.

These are informal calibrations, not theorems; the formal content of every contender is a strict inequality checked by Lean's kernel.

3 Contender 5, exactly

`BusyLean/Contender5.lean` ports `Contender.v` one definition at a time. The port keeps every quirk that can affect the value: raw ill-typed terms are interpreted with `error` defaults; variables use reversed de Bruijn indices; lambdas carry both annotations and cast their body; arrow casts recurse only when `type_eqb` succeeds; the enumeration order is the original one, including right-nested `++`; and the winner is chosen by `maxBy`'s strict update, starting from `t0`. A smoke test reproduces the reference table: $0, 2, 10, 146, 24976$ terms and maxima $0, 0, 1, 2, 3$ at depths 0–4.

Agreement of an independent port. Proposal P5 re-implemented the old language *inside* its larger W-language, with W-aware casts and a plain maximum instead of `maxBy`, and asserted without proof that the result is Contender 5. We prove it:

Theorem 1 (`PortAgreement.largest_eq_oldMaximum`)**.** *For every depth n,*

$$\texttt{C5.largestSTLCNatRecNatOfDepth}\ n \;=\; \texttt{BigNumber.oldMaximum}\ n.$$

The proof is a logical relation between the two interpreters: equality at $\mathbb{N}$, preservation at arrows. That avoids function extensionality. Casts in both directions preserve the relation, including the error branches. The fundamental lemma covers reversed lookup, binder casts and natural recursion. P5's enumeration is the translated image of the original, in the same order, and the `maxBy` winner attains the maximum.

4 Five proposals

Five proposals arrived as archives; `proposals/REVIEW.md` has the details. All were rebuilt on Lean 4.34.0, and the two that had never been compiled (P4, P5) needed small repairs, recorded in separate commits.

	base language	amplifier	informal size
P1	System T	FGH over explicit Veblen-style trees	$f_{\varepsilon_0+\alpha}$, $\alpha < \mathrm{BH}$
P2	T + Brouwer trees + oracles	forest reflection on explicit trees	$f_{\mathrm{BH}\cdot\alpha}$
P3	T + unary oracle	reflection on explicit trees	$f_{\varepsilon_0\cdot\alpha}$
P4	T + Brouwer trees + oracle	ramified reflection	$f_{\mathrm{BH}\cdot\alpha}$
P5	T + W-types at all finite types	two diagonals	$f_{\psi(\Omega_\omega)+\omega^2}$

P5 has by far the strongest base and the weakest amplifier, which suggests combining it with the others' amplifiers.

5 Contender 6

5.1 The object language

The language is P5's. Types are generated by $A, B ::= \mathbb{N} \mid A \to B \mid W\,A$, where $W\,A$ is the type of well-founded trees branching over A. Terms are raw (ill-typed terms default), with natural recursion and a W-recursor into every type. Because A ranges over all types, the language has tree classes of every finite level, $W\mathbb{N}$, $W(W\mathbb{N})$, …, and its calibration is $|L| \approx \psi(\Omega_\omega)$.

5.2 Oracles for free

P5's interpreter resolves variables by oldest-first levels. Evaluating a program in the one-element environment $[\langle \mathbb{N} \to \mathbb{N} \to \mathbb{N}, fs\rangle]$ therefore makes `var 0` denote the oracle family fs everywhere, even under binders. No new syntax is needed, and the smoke test confirms it: `var0 3 4` evaluates to $fs(3,4)$, and so does `(fun y => var0 y y) 5` with the family applied at $(5,5)$.

5.3 Reflection along the language's own trees

```
def reflect (fs : Family) (n : ℕ) : ℕ :=
  n + 1 + max (prefixMax (fun i => fs i n) n)
              (maximum (evalWith fs) (allTerms n))
def stage (seed : Family) : WTree ℕ → ℕ → ℕ
  | .leaf   => reflect seed
  | .node f => reflect (fun i => stage seed (f i))
def treesOfDepth (k : ℕ) : WTree ℕ :=
  .node fun i => defTree (nthD (allTerms k) i .zero)
```

```
def index : WTree ℕ := .node treesOfDepth
def contender_6 : ℕ := stage p5Family index 42
```

At a node the oracle family *is* the family of child stages, so a program may call any child at a computed index (P2's forest reflection). The new ingredient is `index`: its k-th child joins every $W\mathbb{N}$ tree denoted by a program of depth $\leq k$. The hierarchy therefore runs along the language's own ordinals, all of $|L|$, instead of an explicit notation system below BH. By the multiplication heuristic, Contender 6 is about $f_{|L|\cdot|L|}(42)$.

5.4 What is proved

- `contender_5_lt_contender_6`: the exact port of Contender 5 is below Contender 6.
- `p5_lt_contender_6` and `p5Family_lt_finalFamily`: P5 is beaten at every input.
- `defined_tree_eventually_lt`: every tree the language defines is eventually dominated.
- `program_lt_finalFamily`: every program of depth $\leq n$, run with the final oracles, is exceeded.

All of them depend only on `propext`. The definitions check in about 5 s and the proofs in about 6 s. A machine-checked audit shows that the definition never mentions `contender_5` or its winner selection. It also reports, candidly, that P5's seed contains the old language's maximum as a function of depth.

6 Engineering lessons

- **Toolchain drift changes axiom lists.** In Lean 4.34 a wildcard match arm compiles to a `_sparseCasesOn` helper built on `Nat.land`. `Nat.bitwise`'s termination proof uses `propext`, so "axiom-free on 4.19" did not survive the upgrade. The same happens to core `xs[i]?`. Exhaustive matches avoid it.
- **Tactics carry axioms.** `omega`'s certificates use `funext`, and so `Quot.sound`. Two calls were replaced by explicit lemma chains.
- **Never let the unifier normalize a contender.** Writing `iterate f 41 (f 42)` against `iterate f 42 42` made Lean try to evaluate the number. Generalizing the statement first, so that only $42 = 41 + 1$ is checked, fixes it.
- **Prove inequalities by inclusion, not monotonicity.** Reflection is not monotone in its oracle, because a program may test oracle values. Every comparison therefore exhibits an explicit occurrence: a visible oracle index or an enumerated program.

7 Contender 7: impredicative polymorphism

Contender 6 lives below the next collapsing level: everything it does is predicative. The next qualitative jump is *impredicative polymorphism*. The numeric functions definable in Girard's System F on Church numerals are exactly those provably total in second-order arithmetic Z_2, and $\psi(\Omega_\omega)$ is merely the ordinal of Π^1_1-CA$_0$, a small fragment of Z_2.

7.1 Why Lean can host System F

System F has no set-theoretic model in Lean's predicative `Type`: a model would have to interpret $\forall X.A$ as a product over *all* types, including itself. Girard's normalization proof needs much less. It interprets types as *reducibility candidates*, predicates on terms, and quantifies over all candidates. In Lean a candidate is a structure whose main field is $P : \mathrm{Tm} \to \mathrm{Prop}$. A statement "$\forall C : \mathrm{Cand}, \ldots$" is again a proposition, because `Prop` is impredicative. The normalization proof therefore goes through, even though there is no semantic model.

7.2 The formalization

1. **Untyped core** (`SystemF/Syntax`, `Reduction`): de Bruijn terms with an oracle constant `orc`; parallel substitution with its composition algebra; full β-reduction plus the δ-rule $\texttt{orc}\,\overline{n} \to \overline{f(n)}$ on canonical Church numerals; SN as accessibility.

2. **Types** (`Types`): de Bruijn System F types and Curry-style typing of *erased* terms. A flag enables the oracle rule; numerals are typed without it.

3. **Candidates** (`Candidates`): CR1–CR3, where *neutral* means neither a λ nor the oracle (`orc a` can reduce at the head); the arrow construction; the intersection $\forall$; the interpretation $[\![A]\!]_\rho$; its renaming and substitution lemmas, proved as *equalities* of candidates via funext, propext and proof irrelevance; β-closure for λ; and adequacy. The oracle is justified afterwards. Numerals lie in every interpretation of `nat` (adequacy without the oracle rule), so the δ-reduct of $\texttt{orc}\,a$ stays inside the candidate.

Theorem 2 (`F.sn_of_typed`)**.** *Every term typable in System F with the oracle is strongly normalizing under $\beta\delta$, for every oracle function f. The proof uses only* ***`propext`*** *and* ***`Quot.sound`****.*

7.3 From programs to numbers

Programs are Church-style terms (annotated λ, explicit type abstraction and application), so type inference is decidable. `infer_sound` states that inference implies the Curry typing of the erased term. A program's value is computed as follows: check that it is closed of type $\texttt{nat} = \forall X.(X \to X) \to X \to X$; erase it; normalize it by the normal-order step function `nstep`, iterated by well-founded recursion on the SN certificate (the compiler runs this as an ordinary loop); and read the Church numeral. A smoke test shows the pipeline computing: $2^3 \mapsto 8$ and $3^{2^2} \mapsto 81$ by polymorphic instantiation, oracle calls $\texttt{orc}\,\overline{4} \mapsto 40$ for $f(n) = 10n$, and 0 for ill-typed programs.

7.4 The contender

```
def reflectF (f : ℕ → ℕ) (n : ℕ) : ℕ :=
  n + 1 + max (f n) (maximum (fvalue f) (ctmsUpTo n))
def F7 (n : ℕ) : ℕ := reflectF finalFamily n
def contender_7 : ℕ := F7 (F7 42)
```

The oracle is Contender 6's family. System F programs can call it and amplify it with second-order strength. The outer `F7` runs System F at a syntax budget already larger than Contender 6. Proved: `contender_6_lt_contender_7`, `finalFamily_lt_F7` (at every input), and `fvalue_lt_contender_7` (every System F program of depth $\leq$ `F7 42` is exceeded). The audit confirms that the constant `contender_6` itself never occurs in the definition; only Contender 6's family occurs, as the oracle.

Size. By the heuristics of Section 2, Contender 7 is about $f_{|\mathsf{Z}_2|}$ applied at an input beyond Contender 6. Here $|\mathsf{Z}_2|$ is the proof-theoretic ordinal of second-order arithmetic. Nobody has computed it, but it lies far beyond every ordinal named in this article.

8 Contender 8: type operators

System F_ω adds *type operators*. Kinds are $\kappa ::= \star \mid \kappa \to \kappa$; type constructors are simply typed λ-terms over kinds, with $\to$ and $\forall\kappa$ as constants of kind $\star$. Programs may now abstract over type operators. A typical example is $\Lambda F{:}\star \to \star.\, \lambda x{:}F\,\mathtt{nat}.\, x$. The numeric functions definable in F_ω are those provably total in higher-order arithmetic, beyond second-order arithmetic.

8.1 Candidates over kinds

The semantic domain is defined by recursion on kinds: $\mathrm{Sem}(\star) = \mathrm{Cand}$ and $\mathrm{Sem}(\kappa_1 \to \kappa_2) = \mathrm{Sem}(\kappa_1) \to \mathrm{Sem}(\kappa_2)$, with the arrow on the right being an ordinary Lean function type. A type operator therefore *denotes a function on candidates*, and type-level β-reduction is literally function application. The interpretation recurses on syntax at an *expected kind*, returning a default element on kind mismatch. This is the same dynamic-cast idea as the raw interpreters of Contenders 5 and 6, and it keeps the definition structural. Type application records its argument's kind for this purpose. The substitution lemma, the instantiation lemma and the invariance of the interpretation under β-conversion are proved for well-kinded types. The beta case of invariance is one line: function application plus the instantiation lemma.

Theorem 3 (`Fw.sn_of_typed`)**.** *Every term typable in System F_ω with the oracle is strongly normalizing, using only* `propext` *and* `Quot.sound`*.*

8.2 Deciding type equality, soundly

Church-style checking must compare types up to β. Instead of proving a second normalization theorem for the type level, the checker compares *fuel-bounded* normal forms. Every normal-order step is a genuine β-step, so equal results witness joinability, and `Fw.infer_sound` inserts the corresponding conversions. A program needing more fuel is merely rejected. That loses programs but never admits an ill-typed one, and Contender 8 uses the syntax budget as the fuel. The smoke test checks $(\lambda x{:}(\lambda X{:}\star.X)\,\mathtt{nat}.\, x)\,\overline{3} \mapsto 3$ (rejected with fuel 0), and quantification over an operator, $(\Lambda F{:}\star\to\star.\lambda x{:}F\,\mathtt{nat}.x)\,[\lambda X.X]\,\overline{5} \mapsto 5$.

8.3 The contender

```
def family7 (n : ℕ) : ℕ := F7 (F7 n)      -- family7 42 = contender_7
def F8 (n : ℕ) : ℕ := reflectFw family7 n
def contender_8 : ℕ := F8 (F8 42)
```

A first attempt used `F7` itself as the oracle and tried to compare `F7 (F7 42)` with `F8 (F8 42)`. That needs monotonicity of reflection, which fails. Taking Contender 7's whole construction as the family makes the comparison immediate: `family7 42 < F8 42 < F8 (F8 42)`.

9 Contender 9: reflecting F_ω along the W-language's trees

9.1 Why not the Calculus of Constructions

The natural next rung, the Calculus of Constructions, turns out *not* to be a growth-rate jump. In pure CoC, term dependencies can be erased into F_ω (Geuvers and Nederhof), so the numeric functions CoC represents on Church numerals are essentially those of F_ω. This is why Loader's number sits in the F_ω/CoC class, and Contender 8 already lives there. A genuine jump beyond higher-order arithmetic requires *runtime-to-type dependency with large elimination*: types computed by recursion on input numbers, so that a single program uses types of unbounded order. That is the realm of the Calculus of Inductive Constructions, whose normalization needs a realizability model with universes. It is a much larger project, described in the roadmap.

9.2 A multiplicative step

While that project is pending, Contender 6's amplifier applies to any total language with an oracle. Contender 9 reflects F_ω along Contender 6's self-referential index, the join of every tree the W-language defines:

```
def stage9 (seed : Family) : WTree ℕ → ℕ → ℕ
  | .leaf   => reflect9 seed
  | .node g => reflect9 (fun i => stage9 seed (g i))
def F9 (n : ℕ) : ℕ := stage9 (fun _ => family8) index n
def contender_9 : ℕ := F9 (F9 42)      -- family8 n = F8 (F8 n)
```

`reflect9 fs n` exceeds the visible oracle values $fs(i, n)$, $i \le n$, and every F_ω program of depth $\le n$. F_ω's oracle is unary, so the family is packed as `orc` $(2^i(2m+1)) = fs(i, m)$. Programs can compute these codes and so call any child at any input; the theorem `unpair2_code` shows that the decoder inverts the code. The index trees must still come from the W-language: F_ω has no set-theoretic model, so its programs cannot hand trees back to Lean. By the multiplication heuristic, Contender 9 is about $f_{|\mathsf{Z}_\omega|\cdot\psi(\Omega_\omega)^2}$. Proved: `contender_8_lt_contender_9`, `family8_lt_F9` (at every input) and `defined_tree_eventually_lt9`.

9.3 Budgets for a growing stack

Each contender seeds itself with its predecessor's family, so its definition transitively imports the whole history. The full chain took 16–30 s under parallel load on the reference laptop. A fixed budget for the whole stack would eventually be exhausted by history alone. The rules now treat earlier accepted contenders like the precompiled core library: the 15 s/60 s budgets apply to the new contender. Contender 9's definition checks in about 2 s and its proofs in about 3 s, and the full clean build of all contenders takes about 28 s.

10 Contender 10: kind polymorphism

10.1 Unbounded orders without dependent types

Each F_ω program uses finitely many kinds, fixed in its text, and is provably total in higher-order arithmetic of some finite order. To get past that, a single program must reach kinds of *unbounded* order, chosen at run time. Full dependent types with large elimination would do it. A much cheaper

mechanism suffices: types that quantify over *kinds*, $\forall\kappa.A$ (of kind $\star$). A step function

$$g : \forall\kappa.\, B(\kappa) \to B(\kappa \to \star),$$

iterated m times through the impredicatively encoded existential $\exists\kappa.B(\kappa) = \forall Y{:}\star\,.\,(\forall\kappa.\, B(\kappa) \to Y) \to Y$, climbs m orders, where m is a run-time number. A totality proof for such programs needs reducibility candidates for all finite types uniformly, which is beyond higher-order arithmetic.

Consistency. In pure type system terms this adds the rule $(\triangle, \star)$, types quantifying over the sort of kinds, but not $(\triangle, \square)$, kinds quantifying over kinds. The latter would give Girard's System U^-, which is inconsistent by Hurkens' paradox. With only $(\triangle, \star)$, kinds have no binders, so kind variables always range over closed kinds, a countable set. The Lean proof below is semantic, so it certifies normalization, and with it consistency.

10.2 The model

A *kind valuation* θ maps kind variables to closed F_ω kinds, and kind annotations are evaluated under θ. The single new semantic ingredient is

$$[\![\forall\kappa.A]\!]_{\theta,\rho} = \bigcap_{K \text{ closed}} [\![A]\!]_{K\cdot\theta,\rho},$$

a countable intersection of candidates. The renaming and substitution lemmas for kind variables hold unconditionally. Adequacy gains two cases: kind generalization (a fresh kind extends θ, and contexts are kind-shifted) and kind instantiation (choose the evaluated kind).

Theorem 4 (`Fk.sn_of_typed`)**.** *Every term typable in kind-polymorphic System F_ω with the oracle is strongly normalizing, using only* ***`propext`*** *and* ***`Quot.sound`****.*

The Church-style checker extends Contender 8's with kind abstraction and application, again proved sound. A smoke test types $\Lambda\kappa.\Lambda F{:}\kappa.\lambda x{:}\mathtt{nat}.x$ as $\forall\kappa.\forall F{:}\kappa.\,\mathtt{nat} \to \mathtt{nat}$. It runs it at kind $\star$ (giving 4) and at kind $\star \to \star$ with the identity operator (giving 6), and rejects an ill-kinded instantiation.

10.3 The contender

```
def family9 (n : ℕ) : ℕ := F9 (F9 n)       -- family9 42 = contender_9
def F10 (n : ℕ) : ℕ := reflectFk family9 n
def contender_10 : ℕ := F10 (F10 42)
```

Proved: `contender_9_lt_contender_10`, `family9_lt_F10` (at every input), and `fvalue_lt_contender_10`. The new definition modules, four for the language plus the contender, check in about 12 s.

11 Contender 11: reflecting the kind-polymorphic language

Contender 11 applies Contender 9's amplifier to Contender 10's language: forest reflection of kind-polymorphic F_ω along Contender 6's index, with oracle families packed into the unary oracle as before.

```
def F11 (n : ℕ) : ℕ := stage11 (fun _ => family10) index n
def contender_11 : ℕ := F11 (F11 42)      -- family10 n = F10 (F10 n)
```

As with Contender 9, the gain is multiplicative, of about $|\mathrm{F}^{\kappa}_{\omega}| \cdot \psi(\Omega_\omega)^2$, where $\mathrm{F}^{\kappa}_{\omega}$ denotes Contender 10's kind-polymorphic language.

12 A second batch of proposals, and Contender 12

12.1 Five more proposals

Five more archives arrived, all proposing a "Contender 7" on top of this repository's Contender 6, written before Contenders 7–11 existed. All five definitions compile under Lean 4.34. Two proposals compile completely: P6 (operator reflection), whose author had not compiled it, and P7 (critical-point reflection). The other three (P8 ramified worlds, P9 model ramification, P10 semantic mirror) fail in their proofs.

All five amplify the W-language, of strength about $\psi(\Omega_\omega)$. P6 lifts reflection to *transformers* of families and iterates the lift, which exponentiates the index height. P7 stacks explicit Veblen-style critical-point notations above the language-defined index. P8–P10 iterate world or model constructions in which a lower world's whole hierarchy is exposed to the next world's programs. The resulting ordinals range from $|L|^{|L|}$ to Γ-type closures above $|L|$, all far below second-order arithmetic. Informally, the whole batch is vastly smaller than Contender 7 (System F), let alone Contenders 8–11.

12.2 Reusing the generic ideas

Two of the ideas are generic and compose with stronger languages. P6's operator stage needs only that its base transformer be *progressive*: every visible seed value lies below every output. P7's notations apply above any Lean tree. Contender 12 adopts both, with attribution in the source, over the kind-polymorphic F_ω reflection:

```
def fkTransformer : Transformer := fun fs _ n => reflect11 fs n
def tree12 := realize (hyper 42 (encode index))   -- P7 notations
def F12 (n : ℕ) : ℕ :=
  operatorStage fkTransformer tree12 (fun _ => family11) n n   -- P6
def contender_12 : ℕ := F12 (F12 42)
```

The base is progressive by one line (`oracle_lt_reflect11`), and P6's theorem `operatorStage_progressive` then gives `family11_lt_F12` at every input, with no structural facts about `tree12` required. The gain over Contender 11 is again inside the language's ordinal: the index factor grows from $\psi(\Omega_\omega)^2$ to roughly $|L|^{\Gamma\text{-closure above } |L|}$. The world and model constructions of P8–P10 need an object language that manipulates trees as data, which the impredicative languages lack. They are recorded for a future tree-capable language.

13 Contender 13: a predicative universe of operators

13.1 The idea

Kind polymorphism (Contender 10) lets one program reach every *finite* order uniformly. The next step is an object that collects all finite orders at once. Letting kinds quantify over kinds would do it

impredicatively, but that is Girard's System U$^-$, which is inconsistent. The predicative alternative is a new kind `dyn` whose semantic domain is the dependent sum

$$\mathrm{Sem}(\texttt{dyn}) \;=\; \textstyle\sum_{k\ \text{dyn-free}} \mathrm{Sem}(k),$$

which is exactly the type `SVal` of Contender 8's model. It is a universe of all finite-order operators that does not contain itself. Two type-level constructs reach it: $\texttt{pack}\,K\,A : \texttt{dyn}$ for $A : K$, and $\texttt{unpack}\,K\,X : K$ for $X : \texttt{dyn}$, with the conversion $\texttt{unpack}\,K\,(\texttt{pack}\,K\,A) \to A$. Both are restricted to *dyn-free* kinds, a syntactic condition.

Kind variables now come in two sorts. Level-0 variables range over dyn-free kinds and can appear in `pack`/`unpack`. Level-1 variables range over *all* kinds, including `dyn`, $\texttt{dyn} \to \star$, and so on. The existential climbing trick of Contender 10, applied to level-1 variables, therefore reaches orders $\omega + n$ for run-time n.

13.2 The model

Semantic kinds are closed syntactic kinds, which have decidable equality, so the dynamic read `SVal.get` of Contender 6 still works. A decoding `toFw` sends dyn-free closed kinds to F_ω kinds, with $\mathrm{Sem}(K) = \mathrm{Sem}_{F_\omega}(k)$ whenever $\texttt{toFw}\,K = \texttt{some}\,k$. `pack` is `SVal.mk` through that cast, and `unpack` is `SVal.get` through the reverse cast. The lemma `unpack_pack` states that the read-back of a packed value is the value itself, which is what makes the new conversion sound. Level-0 kind variables are evaluated to F_ω kinds and level-1 variables to arbitrary kinds; $\forall\kappa_0$ and $\forall\kappa_1$ are intersections over the respective ranges.

Theorem 5 (`Fd.sn_of_typed`)**.** *Every term typable in dynamic-kind System F_ω with the oracle is strongly normalizing, using only* `propext` *and* `Quot.sound`*.*

The smoke test exercises all of it. A program typable only through $\texttt{unpack}\,\star\,(\texttt{pack}\,\star\,\texttt{nat}) \equiv \texttt{nat}$ evaluates to 3. Level-1 polymorphism instantiated at `dyn` evaluates to 5. Instantiating a level-0 variable with `dyn`, and packing at a kind containing `dyn`, are both rejected.

13.3 The contender

```
def family12 (n : ℕ) : ℕ := F12 (F12 n)     -- family12 42 = contender_12
def F13 (n : ℕ) : ℕ := reflectFd family12 n
def contender_13 : ℕ := F13 (F13 42)
```

Proved: `contender_12_lt_contender_13`, `family12_lt_F13` at every input, and `fvalue_lt_contender_13`. The new definition modules check in about 7.5 s.

13.4 A third batch of proposals

Six more proposals (P11–P16) arrived, written against Contender 10 (and one against Contender 6). Three compile completely (P11, P14, P15). The other three fail in their proofs, all through the same trap: a proof step that makes Lean compare or unfold a term containing a contender-sized number. P13's proof module did not terminate within minutes. Their amplifiers (self-fed iteration counts, crowns above the index, finite diagonal ranks of the autonomous lift) each add about a factor of ω to the index ordinal. Contender 12's tree-indexed operator stage subsumes them, and Contender 13's language exceeds them. P15 independently rediscovered Contender 11's exact construction.

14 Contender 14: the ladder of universes

14.1 The idea

`dyn` collects the operators of every dyn-free kind, but it never contains itself, and nothing collects operators whose kinds mention `dyn`. The predicative answer is a ladder. The level-ℓ language has atomic kinds $\texttt{dyn}_0, \ldots, \texttt{dyn}_{\ell-1}$, and atom $\texttt{dyn}_i$ denotes the `dyn` of the level-i language. Two type-level constructs move operators into and out of an atom:

$$\frac{A : K \qquad K \text{ fits } a}{\texttt{apack}\, a\, K\, A : a} \qquad \frac{X : a \qquad K \text{ fits } a}{\texttt{aunpack}\, a\, K\, X : K} \qquad \texttt{aunpack}\, a\, K\, (\texttt{apack}\, a\, K\, A) \;\longrightarrow\; A,$$

where K *fits* a when K is built from $\star$, $\to$ and atoms strictly older than a. The top-level `dyn` and both sorts of kind variables stay as in Contender 13. So a program can pack an operator of kind $\texttt{dyn}_0 \to \star$ into $\texttt{dyn}_1$ and read it back.

14.2 One normalization proof for every level

Writing a new model for each level would never finish, so the language and its model are *parametric*. The syntax (`SystemFL`) is parametric in a type α of atoms with decidable equality and an order (`AtomOrder`). The model takes a `Universe`: a domain $\mathrm{El}(a)$ for each atom, a packing map from the base kinds that fit a, a reading map, and the law

$$\mathrm{rd}_a\, k\, (\mathrm{pk}_a\, k\, v) = v.$$

Base kinds are $\star$, $\to$ and atoms, interpreted with $\mathrm{Sem}(a) = \mathrm{El}(a)$. `dyn` is the sum over base kinds, and `apack`/`aunpack` are pk/rd through the decoding cast. The read-back law is exactly what the new conversion needs.

Theorem 6 (`FL.sn_of_typed`)**.** *For every universe U, every term typable in the ladder language over U's atoms (with the oracle) is strongly normalizing.*

The ladder itself (`SystemFL/Ladder.lean`) instantiates this theorem at every level:

- atoms are $\texttt{Atom}\, 0 = \texttt{Empty}$ and $\texttt{Atom}\, (\ell + 1) = \texttt{Option}\, (\texttt{Atom}\, \ell)$. The new atom `none` lies above all older atoms $\texttt{some}\, a$;
- `nextUniv` interprets the new atom as `BVal` of the previous universe, i.e. as its `dyn`, and keeps the older atoms;
- packing and reading go through `lower`, which recognizes the level-$(\ell + 1)$ base kinds that avoid the new atom and turns them into level-ℓ base kinds. Semantic domains agree along it, so the read-back law at level $\ell + 1$ reduces to the law at level ℓ.

One engineering point matters. pk receives a proof that the kind fits, so the lowered kind is extracted with `Option.get` rather than a dependent `match`. The law then needs no rewriting under a dependent match, the usual source of ill-typed motives.

14.3 The contender

```
def ladderUpTo (n : ℕ) := ctmsUpTo (Atom.all n) n      -- level-n programs
def reflectFL (f : ℕ → ℕ) (n : ℕ) : ℕ :=
  n + 1 + maxN (f n) (maximum (fvalue n f n) (ladderUpTo n))
def family13 (n : ℕ) : ℕ := F13 (F13 n)     -- family13 42 = contender_13
def F14 (n : ℕ) : ℕ := reflectFL family13 n
def contender_14 : ℕ := F14 (F14 42)
```

The value of a level-ℓ program is computed by the certified normalizer with the SN proof `FL.sn_of_typed` applied to the level-ℓ universe `Univ f`. The reflection diagonalizes *at the meta level*. Input n uses the level-n language, so no fixed rung keeps up with F_{14}. Proved: `contender_13_lt_contender_14`, `family13_lt_F14` at every input, and `fvalue_lt_contender_14`. The smoke test packs λ_:$\mathtt{dyn}_0$.`nat` into $\mathtt{dyn}_1$ and applies its read-back to a packed `nat`, which evaluates to 4. It also confirms that an atom cannot hold its own kind.

The first budget run hit the 15 s definition limit (16.0 s on a loaded machine). Profiling showed the kind checker's soundness proof taking 2.4 s, with 6×6 case splits inside nested matches. Factoring the checker into small result functions (`arrK`, `appK`, `apackK`, ...) with standalone inversion lemmas cut that module from 5.8 s to 3.2 s. The definitions now check in about 10 s.

15 Roadmap

Levels inside the language. Contender 14 fixes the level per program and diagonalizes outside. Level variables, in the style of universe polymorphism, would let one program quantify over rungs and climb a run-time number of them. That gives orders $\omega \cdot n$ uniformly in one program, and the reflection would then diagonalize at ω^2.

Reflection along trees. As with Contenders 9 and 11, reflecting the ladder along the W-language's trees is a cheap multiplicative step.

Large elimination and trees. Runtime-to-type dependency with large elimination remains the route to a CIC-like system. A language able to hand trees back to Lean would unlock the world and model constructions of P8–P10.

16 Status

Contender	idea	informal size
5 (Coq)	System T, depth 42	$f_{<\varepsilon_0}(42)$
6 (Lean)	W-language, reflected along its own trees	$f_{\psi(\Omega_\omega)^2}(42)$
7 (Lean)	System F with Contender 6 as oracle	$f_{\lvert \mathsf{Z}_2 \rvert}(> C_6)$
8 (Lean)	System F_ω with Contender 7 as oracle	$f_{\lvert \mathsf{Z}_\omega \rvert}(> C_7)$
9 (Lean)	F_ω reflected along W-trees	$f_{\lvert \mathsf{Z}_\omega \rvert \cdot \psi(\Omega_\omega)^2}(> C_8)$
10 (Lean)	kind-polymorphic F_ω	beyond higher-order arithmetic
11 (Lean)	10, reflected along W-trees	multiplicative gain over 10
12 (Lean)	10, operator-level reflection (P6, P7 ideas)	exponential index over 11
13 (Lean)	dynamic kind: a universe of operators	uniform orders $\omega + n$
14 (Lean)	ladder of universes, level n at input n	orders $\omega \cdot n$, diagonalized